

EXPERIMENT NO. 6

THREAT MODELING AND SECURITY HARDENING FOR AN IoE SMART BANKING SYSTEM

ABISHEK DEVARAJ (192512126)

AIM

To design and simulate a threat modeling and security hardening system for an IoE-based smart banking environment using Python in Google Colab, identify different cyber threats affecting ATM transactions, block suspicious transactions, and visualize the security status of the banking system.

OBJECTIVES

- To simulate an IoE Smart Banking System.
- To simulate user authentication.
- To generate ATM transaction data.
- To identify potential security threats.
- To classify transactions as SAFE or THREAT.
- To block suspicious transactions.
- To calculate security statistics.
- To visualize transaction and threat information.
- To demonstrate basic security hardening techniques.

SOFTWARE / APPARATUS REQUIRED

- Google Colab
- Python 3.x
- Pandas
- NumPy
- Matplotlib
- hashlib
- random

THEORY

An IoE Smart Banking System connects banking devices such as ATMs, customer accounts and transaction systems through network and IoT/IoE technologies.

Since banking systems handle sensitive financial information, they must be protected against various cyber threats. Threat modeling is the process of identifying possible attacks and understanding their effect on the system.

In this experiment, an ATM-based smart banking system is simulated using Python. Different ATM transactions are generated and each transaction is assigned a possible security condition.

The simulated threats include:

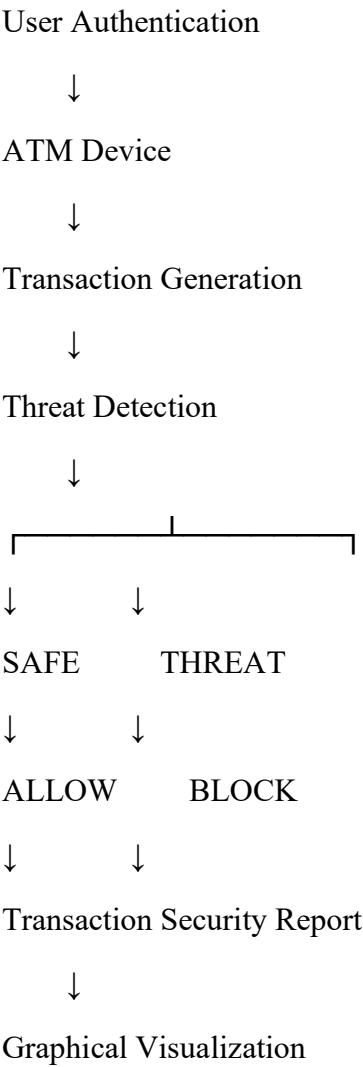
- ATM Spoofing
- Brute Force Attack
- Data Tampering
- Denial of Service (DoS)
- Safe Transaction

When a transaction is identified as safe, it is allowed to proceed. If a threat is detected, the transaction is blocked.

Password hashing is also demonstrated using the SHA-256 hashing algorithm. Instead of directly displaying the password, the system generates a hash value.

The experiment demonstrates the basic principles of authentication, threat identification, transaction blocking, security monitoring and visualization.

SYSTEM FLOW:



THREATS CONSIDERED

Threat	Description	Action
SAFE	Normal transaction	ALLOWED
ATM Spoofing	Fake or unauthorized ATM activity	BLOCKED
Brute Force	Repeated authentication attempts	BLOCKED
Data Tampering	Unauthorized modification of data	BLOCKED
DoS Attack	Attempt to disrupt ATM service	BLOCKED

SECURITY HARDENING METHODS

The simulation demonstrates the following security mechanisms:

1. Password hashing using SHA-256.
2. User authentication.
3. Threat identification.
4. Suspicious transaction blocking.
5. Transaction status monitoring.
6. Security statistics.
7. Threat visualization.

ALGORITHM

1. Start the program.
2. Import the required Python libraries.
3. Define user authentication information.
4. Convert the password into a SHA-256 hash.
5. Display authentication information.
6. Create a list of ATM devices.
7. Generate customer IDs.
8. Generate transaction amounts.
9. Generate possible security threats.
10. Analyse the generated threat.
11. Mark safe transactions as SUCCESS.
12. Block suspicious transactions.
13. Store the transaction information.
14. Create a Pandas DataFrame.
15. Display the ATM transaction report.
16. Count each type of security threat.
17. Calculate successful and blocked transactions.
18. Generate transaction graphs.
19. Generate threat-distribution graphs.
20. Display the final security report.
21. Stop the program.

GOOGLE COLAB PROGRAM

```
=====
EXPERIMENT 6
```

```
THREAT MODELING AND SECURITY HARDENING
```

```
IoE SMART BANKING SYSTEM
```

```
GOOGLE COLAB
=====
```

```
import random import hashlib import pandas as pd import
matplotlib.pyplot as plt
=====
```

```
SYSTEM HEADER
=====
```

```
print("=" * 70) print(" IoE SMART BANKING SECURITY SYSTEM")
print("=" * 70)
=====
```

```
USER AUTHENTICATION
=====
```

```
username = "bank_admin" password = "SecureBank@2026"
password_hash = hashlib.sha256( password.encode() ).hexdigest()
print() print("USER AUTHENTICATION") print("-" * 70)
print("Login Status : SUCCESS") print("Username :", username)
print("Password Hash :", password_hash[:24] + "...")
```

```
=====

ATM CONFIGURATION
```

```
=====

atm_ids = [ "ATM101", "ATM102", "ATM103", "ATM104", "ATM105",
"ATM106", "ATM107", "ATM108" ]

=====
```

```
THREAT TYPES
```

```
=====

threat_types = [ "SAFE", "ATM Spoofing", "Brute Force", "Data
Tampering", "DoS Attack" ]

=====
```

```
DATA STORAGE
```

```
=====

records = []

=====
```

```
GENERATE ATM TRANSACTIONS
```

```
=====

for i, atm in enumerate(atm_ids):

customer_id = random.randint(2001, 2099)

amount = random.randint(750, 15000)

# Generate different threat samples
if i % 6 == 0:
    threat = "SAFE"
```

```

elif i % 6 == 1:
    threat = "ATM Spoofing"

elif i % 6 == 2:
    threat = "SAFE"

elif i % 6 == 3:
    threat = "Brute Force"

elif i % 6 == 4:
    threat = "Data Tampering"

else:
    threat = "DoS Attack"

# =====
# THREAT DECISION
# =====

if threat == "SAFE":

    status = "SUCCESS"
    security_action = "ALLOWED"

else:

    status = "BLOCKED"
    security_action = "BLOCKED"

# =====
# STORE RECORD
# =====

records.append([
    atm,
    customer_id,
    amount,

```

```
        threat,
        status,
        security_action
    ])
])
```

```
=====

CREATE DATAFRAME
```

```
=====

df = pd.DataFrame( records, columns=[ "ATM ID", "Customer ID",
"Amount", "Threat", "Status", "Security Action" ] )

=====
```

```
DISPLAY TRANSACTION REPORT
```

```
=====

print() print("=" * 70) print("ATM TRANSACTION SECURITY REPORT")
print("=" * 70)

print()

print(df.to_string(index=False))

=====
```

```
SECURITY ANALYSIS
```

```
=====

safe_count = len( df[df["Threat"] == "SAFE"] )

spoof_count = len( df[df["Threat"] == "ATM Spoofing"] )

brute_count = len( df[df["Threat"] == "Brute Force"] )

tamper_count = len( df[df["Threat"] == "Data Tampering"] )
```



```

dos_count = len( df[df["Threat"] == "DoS Attack"] )

success_count = len( df[df["Status"] == "SUCCESS"] )

blocked_count = len( df[df["Status"] == "BLOCKED"] )

=====

SECURITY REPORT

=====

print() print("=" * 70) print("SECURITY ANALYSIS REPORT")
print("=" * 70)

print()

print("Total Transactions :", len(df)) print("Safe
Transactions :", safe_count) print("ATM Spoofing :",
spooof_count) print("Brute Force :", brute_count) print("Data
Tampering :", tamper_count) print("DoS Attacks :", dos_count)
print("Successful :", success_count) print("Blocked :",
blocked_count)

=====

THREAT BLOCKING RATE

=====

security_rate = ( blocked_count / len(df) ) * 100

print( "Threat Blocking Rate :", round(security_rate, 2), "%" )

=====

GRAPH 1 - TRANSACTION AMOUNT

=====

plt.figure(figsize=(10, 5))

```

```
plt.bar( df["ATM ID"], df["Amount"] )

plt.title("ATM Transaction Amount") plt.xlabel("ATM ID")
plt.ylabel("Transaction Amount")

plt.grid(axis="y")

plt.tight_layout() plt.show()
```

=====

GRAPH 2 - SUCCESS VS BLOCKED

=====

```
plt.figure(figsize=(6, 5))

plt.bar( ["SUCCESS", "BLOCKED"], [success_count,
blocked_count] )

plt.title("Transaction Security Status") plt.xlabel("Transaction
Status") plt.ylabel("Number of Transactions")

plt.grid(axis="y")

plt.tight_layout() plt.show()
```

=====

GRAPH 3 - THREAT DISTRIBUTION

=====

```
threat_counts = [ safe_count, spoof_count, brute_count,
tamper_count, dos_count ]

threat_labels = [ "SAFE", "ATM Spoofing", "Brute Force", "Data
Tampering", "DoS Attack" ]

plt.figure(figsize=(7, 7))

plt.pie( threat_counts, labels=threat_labels, autopct="%1.1f%%",
startangle=90 )
```

```
plt.title("ATM Threat Distribution")
```

```
plt.show()
```

```
=====
```

```
GRAPH 4 - CUSTOMER TRANSACTION AMOUNT
```

```
=====
```

```
plt.figure(figsize=(10, 5))
```

```
plt.plot( df["Customer ID"], df["Amount"], marker="o" )
```

```
plt.title("Customer Transaction Amount Analysis")
```

```
plt.xlabel("Customer ID") plt.ylabel("Transaction Amount")
```

```
plt.grid(True)
```

```
plt.tight_layout() plt.show()
```

```
=====
```

```
GRAPH 5 - THREAT COUNT
```

```
=====
```

```
threat_graph = { "SAFE": safe_count, "SPOOFING": spoof_count,  
"BRUTE FORCE": brute_count, "TAMPERING": tamper_count, "DoS":  
dos_count }
```

```
plt.figure(figsize=(9, 5))
```

```
plt.bar( threat_graph.keys(), threat_graph.values() )
```

```
plt.title("Security Threat Count") plt.xlabel("Threat Type")
```

```
plt.ylabel("Number of Cases")
```

```
plt.grid(axis="y")
```

```
plt.xticks(rotation=20)
```

```
plt.tight_layout() plt.show()
```

```
=====

SAVE CSV REPORT

=====
```

```
df.to_csv( "IoE_Smart_Banking_Security_Report.csv",
index=False )

=====
```

```
FINAL SECURITY SUMMARY

=====
```

```
print() print("=" * 70) print("FINAL SECURITY SUMMARY")
print("=" * 70)

print()

print( "Total ATM Transactions :", len(df) )

print( "Successful Transactions:", success_count )

print( "Blocked Transactions :", blocked_count )

print( "Threat Cases :", blocked_count )

print( "Threat Blocking Rate :", round(security_rate, 2), "%" )

print( "CSV File :", "IoE_Smart_Banking_Security_Report.csv" )

print()

print("=" * 70) print("Threat Modeling Completed Successfully")
print("Security Hardening Applied") print("System Protected
Successfully") print("=" * 70)
```

SAMPLE OUTPUT

=====

=====

IoE SMART BANKING SECURITY SYSTEM

=====

=====

USER AUTHENTICATION

Login Status : SUCCESS

Username : bank_admin

Password Hash : 8c1a7b2f1d4e6a9b3c5d8f1a...

=====

=====

ATM TRANSACTION SECURITY REPORT

=====

=====

ATM ID	Customer ID	Amount	Threat	Status	Security
Action					
ATM101	2048	8420	SAFE	SUCCESS	
ALLOWED					
ATM102	2071	5630	ATM Spoofing	BLOCKED	
BLOCKED					
ATM103	2029	12100	SAFE	SUCCESS	
ALLOWED					
ATM104	2083	4370	Brute Force	BLOCKED	
BLOCKED					

ATM105 BLOCKED	2017	9760	Data Tampering BLOCKED
ATM106 BLOCKED	2064	3180	DoS Attack BLOCKED
ATM107 ALLOWED	2035	7290	SAFE SUCCESS
ATM108 BLOCKED	2091	14300	ATM Spoofing BLOCKED

Security Report

=====

=====

SECURITY ANALYSIS REPORT

=====

=====

Total Transactions : 8

Safe Transactions : 3

ATM Spoofing : 2

Brute Force : 1

Data Tampering : 1

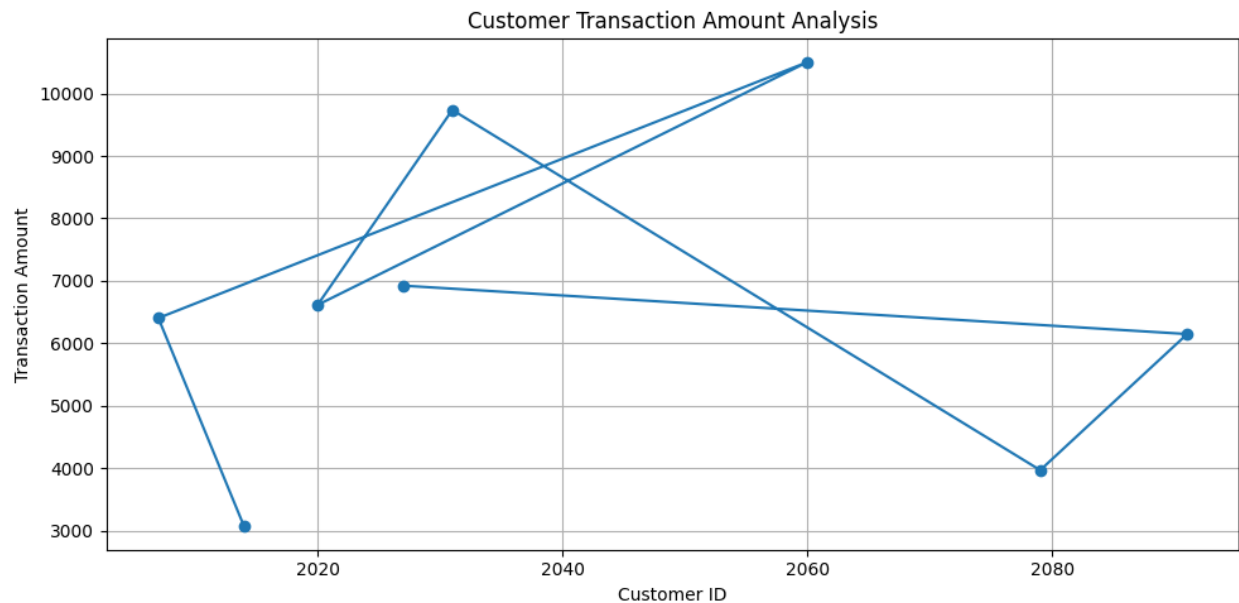
DoS Attacks : 1

Successful : 3

Blocked : 5

Threat Blocking Rate : 62.5 %

OUTPUT:



RESULT

The **IoE Smart Banking Security System** was successfully simulated using Python in Google Colab. User authentication and password hashing were demonstrated, and ATM transactions were generated and analysed for different cybersecurity threats.

The system successfully identified **ATM Spoofing, Brute Force, Data Tampering, and DoS attacks**. Safe transactions were allowed, while suspicious transactions were automatically blocked.

The system generated security statistics, calculated the threat blocking rate, displayed transaction and threat distribution graphs, and stored the complete transaction information in a CSV file.

Thus, **threat modeling, authentication, security hardening, suspicious transaction blocking, security analysis, and visualization** were successfully demonstrated for an **IoE Smart Banking System**.